`two_sylow_search` on `fp_backend::Fp64`

Before and after the branch-free selections: results report, 2026-10-17

Study: `research/two_sylow_fp64_20261017/`; protocol `PROTOCOL.md` frozen at `e71198313` before the first run. **Run report:** `reports/run-20261017T0018Z/run_report.pdf`. **Follows:** `research/fp64_branchfree_20261016/` (PR #2034), whose "Next" item 1 asked whether a latency-bound user of `fp_backend::Fp64` relies on the predictable `sub` chains that the change slowed.

Scope. Wall time of one production module's `Fp64` instantiation. No counted cost changes; not an index-calculus result; no gap-to-rho row.

1. Findings

1. **No detected gain, no slowdown beyond the bound.** Across the six arms (two families × three search arms) the candidate/baseline ratio is 0.98–1.02 in the first pair of runs and 1.02–1.08 in the second (S1 PASS, worst 1.083 against 1.10). Five arms show no detected effect; `tsy.z2xz2.fast-chi` is classed slower (1.023, 1.083).
2. **The second pair reads high for unchanged code too.** Every `tsy` arm's second ratio is above its first, and the candidate's second run (B2) is the noisiest of the four (Figure 1). In the same pair, the unchanged `FpWide<1>` kernel at B ≥ 4096 reads 0.991–1.018 (C1 PASS, max deviation 0.044). Four of the 20 memory-read arms, whose code is identical in both binaries, were also classed slower. Part of the second pair's 2–8% is host drift, not the change.
3. **The comparison sees what it should.** The positive control, the `fp64` rho kernel, is 0.441× or less of baseline at every B ≥ 4096 (K1 PASS), replicating PR #2034. Gate G: both binaries computed the same search (identical depth sums, point counts and per-arm checksums over 2,000 draws per family, in all four runs).
4. **Why no gain.** `Fp64` has no specialised `HalvingField` methods. A draw is dominated by `pow` (square roots and `(p − 3)/4` powers, dependent `mul`/`sqr` chains) and `jacobi_u64`, neither of which the change touches. The dependent `mul` and `sqr` latency arms measured in the same runs are unchanged (`fp64.mul.latency` 1.007, 1.001 `fp64.sqr.latency` 0.999, 0.988). The dependent `sub` chain is again slower (`fp64.add.latency` 1.12, 1.17), as in PR #2034. The search's handful of `add`/`sub` per level is not enough for that to move a draw beyond the noise.

Reading. PR #2034's change is neutral for `two_sylow_search` on `fp_backend::Fp64` within this host's resolution: no arm slowed by 10%, and the first pair (the less noisy one) is within ±2.3%. Production runs at P-192 use `FpP192`, which the change does not touch.

Limits. Host L0 without a PMU; one prime ($2^{63} - 25$, the largest prime $\equiv 3 \bmod 4$ below 2^{63}); record depth 6; baseline-ISA builds only.

2. Claims

id	statement	value	threshold	verdict
S1	max(r1, r2) over 6 arms tsy.* (batch ≥ 0)	1.0832	1.1	PASS
K1	max(r1, r2) over 6 arms kernel.fp64.* (batch ≥ 4096)	0.4415	0.85	PASS
C1	max |r_k − 1| over 6 arms kernel.w1.* (batch ≥ 4096)	0.0445	0.1	PASS
G	gate lines identical in all four runs	—	—	PASS

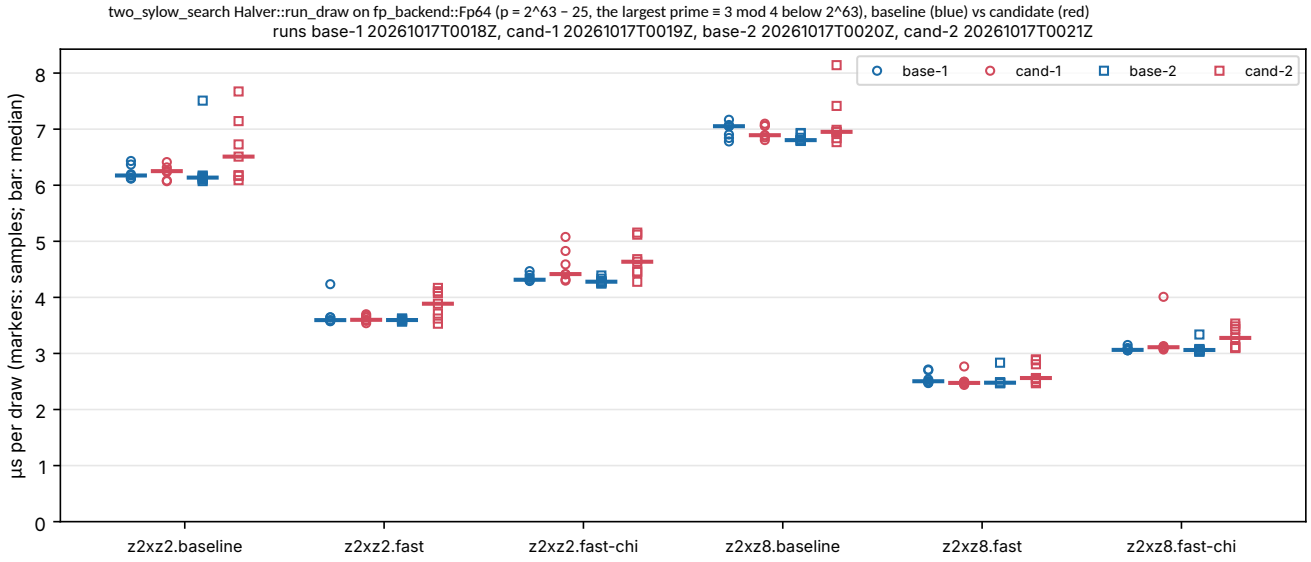


Figure 1: Every sample of the six `two_sylow_search` arms, µs per draw, in the four runs.

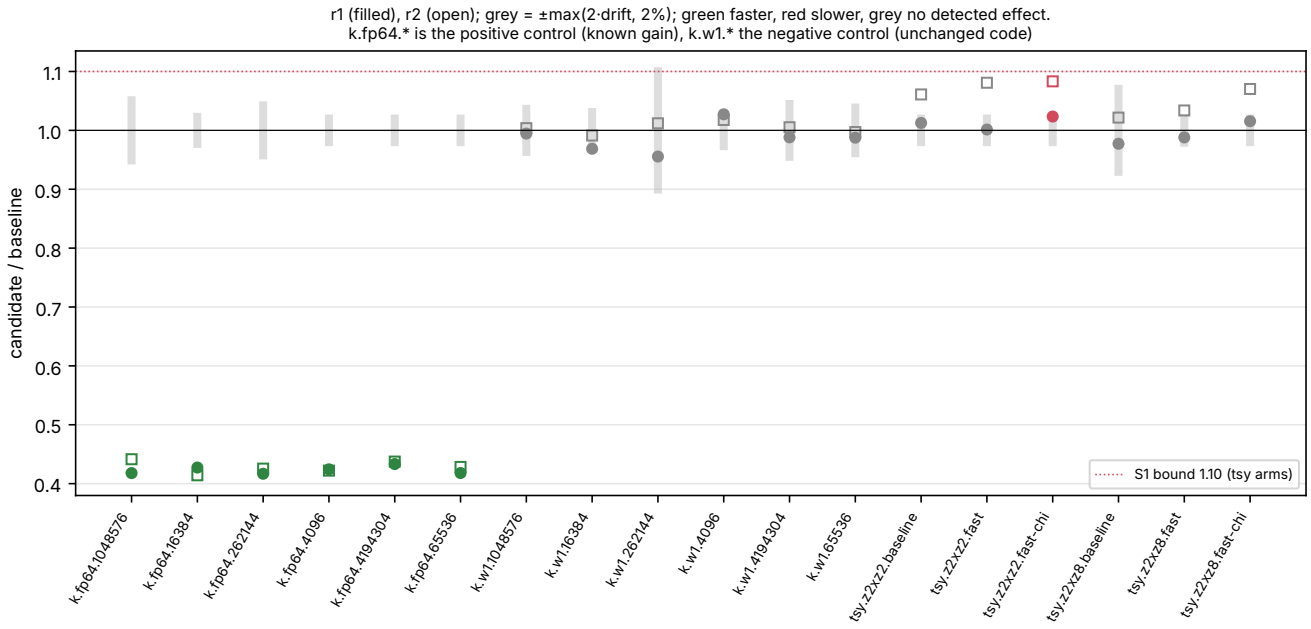


Figure 2: Candidate/baseline ratios of the `tsy` arms and of the kernel controls (positive: `fp64`, negative: `w1`) at B ≥ 4096.